

PID Algorithm Basics

Introduction

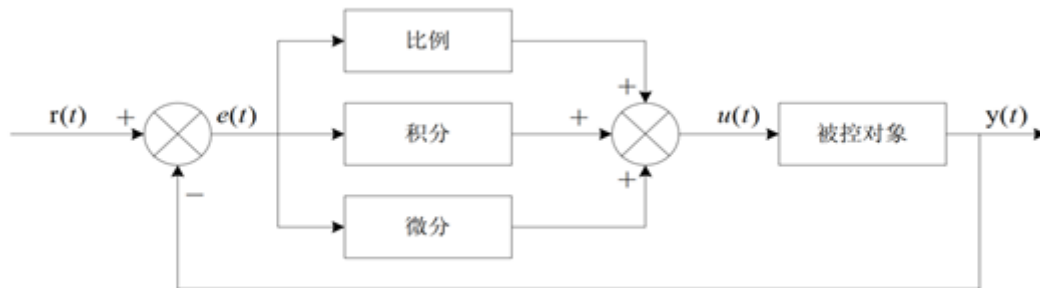
PID, which stands for Proportional Integral Derivative, is a control algorithm that performs proportional, integral, and differential operations on the input deviation, and the superimposed result of these operations is used to control the actuator. The formula is as follows:

$$u(t) = K_p[e(t) + \frac{1}{T_i} \int_0^t e(t)dt + Td\frac{de(t)}{dt}]$$

It consists of three parts:

- P is proportion, which is the input deviation multiplied by a coefficient;
- I is integral, which performs integral operations on the input deviation;
- D is derivative, which performs derivative operations on the input deviation.

The following diagram shows a basic PID controller:



(1) Proportional Part

The mathematical expression for the proportional part is: $K_p \cdot e(t)$

In an analog PID controller, the proportional action responds immediately to deviation. Once deviation occurs, the controller immediately generates control action to direct the control quantity toward reducing the deviation. The strength of the control action depends on the proportional coefficient - the larger the proportional coefficient, the stronger the control action, the faster the transition process, and the smaller the static deviation of the control process; however, if it is too large, it can easily cause oscillation and destroy system stability. Therefore, the proportional coefficient must be chosen appropriately to achieve minimal transition time, small static error, and stability.

Advantages: Adjusting the open-loop proportional coefficient of the system improves steady-state accuracy, reduces system inertia, and speeds up response.

Disadvantages: Using only the P controller, an overly large open-loop proportional coefficient not only increases the system overshoot but also reduces the system stability margin, or even causes instability.

(2) Integral Part

The mathematical expression for the integral part is:

$$\frac{K_p}{T_i} \int_0^t e(t)dt$$

The larger the integral constant, the weaker the accumulation effect of the integral; in this case, the system will not oscillate during transition; however, increasing the integral constant slows down the process of eliminating static error, and it takes longer to eliminate deviation, but it can reduce overshoot and improve system stability. When T_i is small, the integral action is stronger, and oscillation may occur during the system transition, though the time required to eliminate deviation is shorter. Therefore, T_i must be determined based on specific control requirements.

Advantages: Eliminates steady-state error.

Disadvantages: The addition of the integral controller affects system stability and reduces the stability margin.

(3) Derivative Part

The mathematical expression for the derivative part is:

$$K_p * T_d \frac{de(t)}{dt}$$

The derivative action prevents deviation from changing. It performs control based on the trend of deviation (rate of change). The faster the deviation changes, the larger the output of the derivative controller, and it can make corrections before the deviation value increases. The introduction of derivative action helps reduce overshoot, overcome oscillation, and stabilize the system, especially beneficial for high-order systems, as it speeds up system tracking. However, derivative action is sensitive to noise in input signals, so for systems with significant noise, derivative is generally not used, or input signals are filtered before derivative takes effect. The derivative action is determined by the derivative time constant T_d . The larger T_d , the stronger its inhibition of deviation change; the smaller T_d , the weaker its resistance to deviation change. The derivative part clearly plays a significant role in system stability. Appropriate selection of the derivative constant T_d can optimize the derivative effect.

Advantages: Speeds up system response, reduces overshoot and oscillation, and has a "predictive" effect on dynamic processes.

Selection of PID Algorithm

Digital PID control algorithms can be classified into positional PID and incremental PID control algorithms. Before deciding which PID algorithm to use, we should first understand its principles:

- Positional PID Algorithm:

$$u(k) = K_p e(k) + K_I \sum_{i=0} e(i) + K_D [e(k) - e(k-1)]$$

$e(k)$: User-set value (target value) - current state value of the controlled object

Proportional P: $e(k)$

Integral I: $\sum e(i)$ - accumulation of error

Derivative D: $e(k) - e(k-1)$ current error - previous error

In other words, positional PID is the deviation between the current actual position of the system and the desired target position, performing PID control.

Because there is error integration $\sum e(i)$, which accumulates continuously, the current output $u(k)$ is related to all past states, using the accumulated error value; (error e will have error accumulation), the output $u(k)$ corresponds to the actual position of the actuator, and once the control output is incorrect (the current state value of the controlled object has a problem), significant changes in $u(k)$ will cause significant changes in the system. Additionally, when the integral term of positional PID reaches saturation, the error will continue to accumulate under the integral action. Once the error begins to reverse, the system needs a certain time to exit the saturation zone. Therefore, when $u(k)$ reaches maximum and minimum, integral action should be stopped, and there should be integral limiting and output limiting. So when using positional PID, we generally directly use PD control. Positional PID is suitable for control objects where the actuator does not have integral components, such as steering gear and balance car upright control and temperature control systems.

Advantages: Positional PID is a non-recursive algorithm that can directly control actuators (such as balance cars), and the $u(k)$ value corresponds one-to-one with the actual position of the actuator (such as the current angle of the car), so it can be well applied in control objects where the actuator does not have integral components.

Disadvantages: Each output is related to past states, and during calculation, $e(k)$ must be accumulated, which requires a large amount of computation.

- Incremental PID:

$$\Delta u(k) = u(k) - u(k-1) = K_P[e(k) - e(k-1)] + K_I e(k) + K_D[e(k) - 2e(k-1) + e(k-2)]$$

Proportional P: $e(k) - e(k-1)$ current error - previous error

Integral I: $e(k)$ error

Derivative D: $e(k) - 2e(k-1) + e(k-2)$ current error - 2*previous error + error before previous

From the formula, incremental PID clearly shows that once K_P , T_I , and T_D are determined, the control increment can be calculated using the deviation of the three consecutive measurement values before and after. The resulting control quantity $\Delta u(k)$ corresponds to the increment of recent position errors, not the deviation from the actual position. There is no error accumulation, meaning incremental PID does not require accumulation. The determination of control increment $\Delta u(k)$ is only related to the three most recent sampling values, making it easy to obtain good control effects through weighted processing, and when a problem occurs in the system, incremental PID will not seriously affect system operation.

Summary: Incremental PID takes the increment of positional PID. At this point, the controller output is the difference between the position values calculated at two consecutive sampling instants. The result is an increment, meaning on the basis of the previous control quantity, an increase is needed (a negative value means decrease).

Advantages: ① When a mistaken action occurs, the impact is small, and erroneous data can be removed through logical judgment when necessary.

② Low impact during manual/automatic switching, facilitating disturbance-free switching. When a computer fails, the original value can still be maintained.

③ No accumulation is required in the formula. The determination of control increment $\Delta u(k)$ is only related to the three most recent sampling values.

Disadvantages: ① Large integral truncation effect, there is steady-state error;

② Significant impact of overflow. Some controlled objects do not work well with incremental;

- Differences Between Incremental and Positional

① The incremental algorithm does not require accumulation, and the determination of control quantity increment is only related to several recent deviation sampling values, so the calculation error has a smaller effect on the control quantity calculation. The positional algorithm uses the accumulated value of past deviations, which easily generates larger accumulated errors.

② The incremental algorithm yields the increment of the control quantity. For example, in valve control, only the part of valve opening change is output, so the impact of mistaken action is small; when necessary, logical judgment can limit or prohibit this output, which will not seriously affect system operation. The output of positional directly corresponds to the object output, so it has a greater impact on the system.

③ Incremental PID control outputs the control quantity increment and has no integral action, so this method is suitable for actuators with integral components, such as stepper motors, while positional PID is suitable for actuators without integral components, such as electro-hydraulic servo valves.

④ When performing PID control, positional PID requires integral limiting and output limiting, while incremental PID only needs output limiting. Positional PID and incremental PID are just two implementation forms of digital PID control algorithms, with identical essence. The main difference is the different storage method of the integral term. Positional PID stores the integral term separately, while incremental PID stores the integral term as part of the output. Online various players also have their own unique insights and views on the use of positional and incremental. Specifically, it depends on which algorithm is suitable for our specific application scenario.

PID Parameter Tuning

There are many methods for selecting PID controller parameters, such as trial-and-error method, critical proportioning method, extended critical proportioning method, etc. However, for PID control, parameter selection is always a very tedious work, and requires continuous adjustment to obtain a satisfactory control effect. Based on experience, the general steps for determining PID parameters are as follows:

- Determine proportional coefficient K_p

When determining the proportional coefficient K_p , first remove the integral and derivative terms of PID, set $T_i=0$, $T_d=0$, making it

Pure proportional adjustment. Set the input to 60%~70% of the maximum output value allowed by the system, and gradually increase the proportional coefficient K_p from 0 until the system oscillates; then reverse, gradually decrease the proportional coefficient K_p until the oscillation disappears. Record the proportional coefficient K_p at this time, and set the PID proportional coefficient K_p to 60%~70% of the current value.

- Determine integral time constant T_i

After determining the proportional coefficient K_p , set a larger integral time constant T_i , then gradually decrease T_i until the system oscillates, then reverse, gradually increase T_i until the oscillation disappears. Record T_i at this time, and set the PID integral time constant T_i to 150%~180% of the current value.

- Determine derivative time constant T_d

The derivative time constant T_d is generally not set, just set it to 0, at this time PID adjustment becomes PI adjustment. If it needs to be set, it is the same method as determining K_p , taking 30% of the value when there is no oscillation.

- System no-load and load joint debugging

Fine-tune PID parameters until performance requirements are met.

Of course, this is my personal debugging method, which may not be suitable for everyone and every environment, just for your reference; however, there is also a classic trial-and-error rhyme for debugging PID circulating online, which I will share for your reference:

Parameter tuning finds the best, from small to large in order.

First proportional then integral, finally add derivative.

Curve oscillates very frequently, increase proportional dial.

Curve drifts and makes a big bend, decrease proportional dial.

Curve deviates and recovers slowly, lower integral time.

Curve fluctuates period long, increase integral time again.

Curve oscillates frequency fast, first lower derivative.

Large error with slow fluctuation, increase derivative time.

Ideal curve two waves, high front and low back four to one.

First observe then adjust more analysis, adjustment quality will not be low.

PID is a control algorithm consisting of proportion (P), integration (I), and derivation (D). It does not necessarily require all three algorithms; it can also be PD, PI, or even just P algorithm control. My previous simplest idea for closed-loop control was only P control, feeding the current result back, then subtracting from the target; if positive, slow down; if negative, speed up. Of course, this is the simplest closed-loop control algorithm, so I return to the summary of positional and incremental in the previous section. Specifically, it depends on our current control environment, because each control system is different, and the parameters that can make our system achieve the most stable effect are naturally OK.